

Variables and Constraints and Objectives

The lowest-level building blocks for Gurobi models are variables, constraints, and objectives. While each has a clean mathematical definition, linear and integer programming aren't performed in exact arithmetic, so computed results can sometimes deviate from these clean definitions. This section discusses the use of and restrictions on these basic building blocks.

10.1 Variables

Decision variables capture the results of the optimization. In a feasible solution, the computed values for the decision variables satisfy all of the model constraints. Some of these constraints are associated with individual variables (e.g., variable bounds), while others capture relationships between variables. We'll first consider the different types of decision variables that can be added to a Gurobi model, and the implicit and explicit constraints associated with these variable types.

Before starting, we should point out one important thing about the variables in a mathematical programming model: their computed solution values will only satisfy bounds *to tolerances*, meaning that a variable may violate its stated bounds. Mathematical programming is fundamentally built on top of linear algebra and in particular on the numerical solution of systems of linear equations. These linear systems are solved using finite-precision arithmetic, which means that small errors are unavoidable. For some models, large errors are unavoidable too; we'll return to that topic later in this section.

The available variables types are [continuous](#), [general integer](#), [binary](#), [semi-continuous](#), and [semi-integer](#).

Continuous Variables

The simplest and least constrained of the available variable types is the continuous variable. This variable can take any value between its lower and upper bound. In mathematical programming, the convention is that variables are non-negative unless stated otherwise, so if you don't explicitly provide bounds for a variable, you should assume that the lower bound is 0 and the upper bound is infinite.

The Gurobi APIs provides a symbolic constant to allow you to indicate that a bound is infinite (`GRB_INFINITY` in C and C++, `GRB.INFINITY` in C#, Java, and Python). A variable can have an infinite upper bound, an infinite lower bound (negative infinity), or both. A variable with infinite upper and lower bounds is referred to as a *free variable*. Any bound larger than $1e30$ is treated as infinite.

As noted earlier, variables may violate their bounds by tolerances. In the case of variable bounds, the relevant tolerance value is the [FeasibilityTol](#). You can reduce the value of this tolerance parameter, but due to numerical errors it may not be possible to achieve your desired accuracy.

General Integer Variables

General integer variables are more constrained than continuous variables. In addition to respecting the specified lower and upper bounds, integer variables also take integral values.

Due to the limitations of finite-precision arithmetic, integer variables will often take values that aren't exactly integral. The magnitude of the allowed integrality violation is controlled by the [IntFeasTol](#) parameter. You can tighten this parameter to reduce the magnitude of these integrality violations, but the cost of solving the optimization problem may increase significantly as a result.

The fact that modern computers represent integer values using 32-bit values places some restrictions on the range of an integral variable. Specifically, the largest and smallest bounds that can be placed on an integer variable are $\pm 2,000,000,000$. Furthermore, integer variables with infinite bounds actually have these values as their implicit bounds. A solution is not considered feasible unless all integer variables take values that satisfy these bounds.

Binary Variables

Binary variables are the most constrained variable type that can be added to your model. A binary variable takes a value of either 0 or 1.

Again, due to the limitations of finite-precision arithmetic, binary variables will often take values that aren't exactly integral. The magnitude of the allowed integrality violation is controlled by the [IntFeasTol](#) parameter.

Semi-Continuous and Semi-Integer Variables

You can also add semi-continuous or semi-integer variables to your model. A semi-continuous variable has the property that it takes a value of 0, or a value between the specified lower and upper bounds. A semi-integer variable adds the additional restriction that the variable also take an integral value.

Again, these variables may violate these restrictions up to tolerances. In this case, the relevant tolerance is [IntFeasTol](#) (even for semi-continuous variables).

10.2 Constraints

A constraint in Gurobi captures a restriction on the values that a set of variables may take. The simplest example is a linear constraint, which states that a linear expression on a set of variables take a value that is either less-than-or-equal, greater-than-or-equal, or equal to another linear expression. More complicated constraints are also supported, including quadratic constraints (e.g., $x^2 + y^2 \leq z^2$), logical constraints (e.g., logical AND on binary variables, if-then, etc.), and a few non-linear functions (e.g., $y = \sin(x)$).

We now consider a few more details about [linear](#), [SOS](#), [quadratic](#) (both convex and non-convex), and [general](#) constraints. General constraints are the catch-all we use for the constraint types that don't fit in the other categories.

Recall that Gurobi works in finite-precision arithmetic, so constraints are only satisfied *to tolerances*. Tolerances can be tightened to reduce such violations, but there are limits to how small the violations can be -- errors are inherent in floating-point arithmetic. This point will be reiterated in several places in this section.

Linear Constraints

A linear constraint allows you to restrict the value of a linear expression. For example, you may require that any feasible solution satisfies the constraint $3x + 4y \leq 5z$. Note that the matrix-oriented Gurobi APIs (C, MATLAB, and R) require the right-hand side of a linear constraint to be

a constant, while the object-oriented APIs (C++, Java, .NET, and Python) allow arbitrary linear expressions on both sides of the comparator.

The computed solution should satisfy the stated constraint to within [FeasibilityTol](#) (although it may not in cases of numerical ill-conditioning -- we'll discuss this shortly).

Gurobi supports a limited set of comparators. Specifically, you can constrain an expression to be less-than-or-equal, greater-than-or-equal, or equal another. We do not support strict less-than, strict greater-than, or not-equal comparators. While these other comparators may seem appropriate for mathematical programming, we exclude them to avoid potential confusion related to numerical tolerances. Consider a simple example of a strict inequality constraint on a pair of continuous variables: $x > y$. How large would $x - y$ need to be in order to satisfy the constraint? Rather than trying to embed a subtle and potentially confusing strategy for handling such constraints into the solver, we've chosen not to support them instead.

SOS Constraints

A Special-Ordered Set, or SOS constraint, is a highly specialized constraint that places restrictions on the values that variables in a given list can take. There are two types of SOS constraints. In an SOS constraint of type 1 (an SOS1 constraint), at most one variable in the specified list is allowed to take a non-zero value. In an SOS constraint of type 2 (an SOS2 constraint), at most two variables in the specified, ordered list are allowed to take a non-zero value, and those non-zero variables must be contiguous in the list. The variables in an SOS constraint can be continuous, integer, or binary.

Again, tolerances play an important role in SOS constraints. Specifically, variables that take values less than [IntFeasTol](#) (in absolute value) are considered to be zero for the purposes of determining whether an SOS constraint is satisfied.

An SOS constraint is described using a list of variables and a list of corresponding weights. While the weights have historically had intuitive meanings associated with them, we simply use them to order the list of variables. The weights should be unique. This is especially important for an SOS2 constraint, which relies on the notion of *contiguous* variables. Since the variables in the SOS are ordered by weight, contiguity becomes ambiguous when multiple variables have the same weight.

It is often more efficient to capture SOS structure using linear constraints rather than SOS constraints. The optimizer will often perform this reformulation automatically. This is controlled with four parameters: [PreSOS1BigM](#), [PreSOS1Encoding](#), [PreSOS2BigM](#) and [PreSOS2Encoding](#).

The reformulation adds constraints of the form $x \leq Mb$, where x is the variable that participates in the SOS constraint, b is a binary variable, and M is an upper bound on the value of variable x . Large values of M can lead to numerical issues. The two parameters [PreSOS1BigM](#) and [PreSOS2BigM](#) control the maximum value of M that can be introduced by this reformulation. SOS constraints that would require a larger value aren't converted. Setting one of these parameters to 0 disables the corresponding reformulation.

Additionally, there are several known integer formulations for SOS1 and SOS2 constraints. These reformulations differ in the number of variables that they introduce to the problem, in the complexity of the resulting LP relaxations, and in their properties in terms of branching and cutting planes. The two parameters [PreSOS1Encoding](#) and [PreSOS2Encoding](#) control the choice of the reformulation performed.

Quadratic Constraints

A quadratic constraint allows you to restrict the value of a quadratic expression. For example, you may require that any feasible solution satisfy the constraint $3x^2 + 4y^2 + 5z \leq 10$. Note that the matrix-oriented Gurobi APIs (C, MATLAB, and R) require the right-hand side of a quadratic constraint to be a constant, while the object-oriented APIs (C++, Java, .NET, and Python) allow arbitrary quadratic expressions on both sides of the comparator.

The computed solution should satisfy the stated constraint to within [FeasibilityTol](#). Quadratic constraints are often much more challenging to satisfy than linear constraints, so tightening the parameter may increase runtimes dramatically.

Gurobi can handle both convex and non-convex quadratic constraints. However, there are some subtle and important differences in how the different constraint types are handled. In general, it is much easier to solve a model whose constraints all have convex feasible regions. It is actually quite difficult to recognize all such cases, but the following forms are always recognized:

- $x^T Q x + q^T x \leq b$, where Q is Positive Semi-Definite (PSD)
- $x^T Q x \leq y^2$, where Q is Positive Semi-Definite (PSD), x is a vector of variables, and y is a non-negative variable (a Second-Order Cone constraint, if $Q = I$, identity matrix)
- $x^T Q x \leq yz$, where Q is Positive Semi-Definite (PSD), x is a vector of variables, and y and z are non-negative variables (a rotated Second-Order Cone constraint, if $Q = I$, identity matrix)

To be more precise, a quadratic constraint will always be recognized as convex if presolve is able to transform it into one of these forms. Note that if all quadratic terms in a quadratic constraint contain at least one binary variable, then presolve will always be able to transform it to a convex form.

Why distinguish between convex and non-convex quadratic constraints? In some situations you may know that your problem should be convex, and thus it may be a sign of a modeling error if your model isn't recognized as such. To avoid accidentally solving a much harder problem than you may have intended, you can set the [NonConvex](#) parameter to either 0 or 1. In the default setting of -1 or if the [NonConvex](#) parameter is set to 2, Gurobi will accept arbitrary quadratic constraints and attempt to solve the resulting model using the appropriate algorithm.

Note that other non-convex quadratic solvers often only find locally optimal solutions. The algorithms in Gurobi explore the entire search space, so they provide a globally valid lower bound on the optimal objective value, and given enough time they will find a globally optimal solution (subject to tolerances).

We would like to note a subtle point here regarding terminology. A quadratic constraint that involves only products of disjoint pairs of variables is often called a *bilinear constraint*, and a model that contains bilinear constraints is often called a *bilinear program*. Bilinear constraints are a special case of non-convex quadratic constraints, and the algorithms Gurobi uses to handle the latter are also well suited to solving bilinear programming problems.

General Constraints

The previously-described constraints are typically (but not always) handled directly by the underlying optimization algorithms. Gurobi includes an additional set of higher-level constraints, which

we collectively refer to as *general constraints*, that require special handling. We think of these as belonging to two types: [simple constraints](#) and [function constraints](#)

Simple general constraints are a modeling convenience. They allow you to state fairly simple relationships between variables (min, max, absolute value, logical OR, etc.). Techniques for translating these constraints into lower-level modeling objects (typically using auxiliary binary variables and linear or SOS constraints) are well known and can be found in optimization modeling textbooks. By automating these translations and removing the need for the modeler to perform them, the hope is that these simple general constraints will allow you to write more readable and more maintainable models.

Function constraints allow you to state much more complex relationships between variables; you can require that $y = f(x)$, where x and y are Gurobi decision variables and $f()$ is chosen from a predefined list of nonlinear functions. The common theme among these functions is that there is no simple way to restate the associated constraints using the primitive objects (like integer variables and linear constraints) that a standard MIP solver wants to work with. Alternate algorithms are required, and the resulting models are typically much more difficult to solve than models that do not contain these constraints.

Simple General Constraints

Gurobi supports the following simple general constraints, each with its own syntax and semantics:

- **MAX constraint:** The constraint $r = \max\{x_1, \dots, x_k, c\}$ states that the *resultant variable* r should be equal to the maximum of the *operand variables* $x_1, \dots, x_k$ and the *constant* c . For example, a solution ($r = 3, x_1 = 2, x_2 = 3, x_3 = 0$) would be feasible for the constraint $r = \max\{x_1, x_2, x_3, 1.7\}$ because 3 is indeed the maximum of 2, 3, 0, and 1.7.
- **MIN constraint:** Similar to a MAX constraint, the constraint $r = \min\{x_1, \dots, x_k, c\}$ states that the *resultant variable* r should be equal to the minimum of the *operand variables* $x_1, \dots, x_k$ and the *constant* c .
- **ABS constraint:** The constraint $r = \text{abs}\{x\}$ states that the *resultant variable* r should be equal to the absolute value of the *operand variable* x . For example, a solution ($r = 3, x = -3$) would be feasible for the constraint $r = \text{abs}\{x\}$.
- **AND constraint:** The constraint $r = \text{and}\{x_1, \dots, x_k\}$ states that the binary *resultant variable* r should be 1 if and only if all of the binary *operand variables* $x_1, \dots, x_k$ are equal to 1. For example, a solution ($r = 1, x_1 = 1, x_2 = 1, x_3 = 1$) would be feasible for the constraint $r = \text{and}\{x_1, x_2, x_3\}$. Note that any involved variables that are not already binary are converted to binary.
- **OR constraint:** Similar to an AND constraint, the constraint $r = \text{or}\{x_1, \dots, x_k\}$ states that the binary *resultant variable* r should be 1 if and only if at least one of the binary *operand variables* $x_1, \dots, x_k$ is equal to 1. Note that any involved variables that are not already binary are converted to binary.
- **NORM constraint:** The constraint $r = \text{norm}\{x_1, \dots, x_k\}$ states that the *resultant variable* r should be equal to the vector norm of the *operand variables* $x_1, \dots, x_k$. A few options are available: the 0-norm, 1-norm, 2-norm, and infinity-norm.

- **INDICATOR constraints:** An indicator constraint $y = f \rightarrow a^T x \leq b$ states that if the binary *indicator variable* y is equal to f in a given solution, where $f \in \{0, 1\}$, then the linear constraint $a^T x \leq b$ has to be satisfied. On the other hand, if $y \neq f$ (i.e., $y = 1 - f$) then the linear constraint may be violated. Note that the sense of the linear constraint can also be $=$ or $\geq$; refer to [this earlier section](#) for a more detailed description of linear constraints. Note also that declaring an INDICATOR constraint implicitly declares the indicator variable to be of binary type.
- **Piecewise-linear constraints:** A piecewise-linear constraint $y = f(x)$ states that the point (x, y) must lie on the piecewise-linear function $f()$ defined by a set of points $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. Refer to the description of [piecewise-linear objectives](#) for details of how piecewise-linear functions are defined.

Note that adding any of these constraints to an otherwise continuous model will transform it into a MIP

As stated above, each general constraint has an equivalent MIP formulation that consists of linear and SOS constraints, and possibly auxiliary variables. Thus, you could always model such constraints yourself without using a Gurobi general constraint. For example, the MAX constraint $r = \max\{x_1, \dots, x_k, c\}$ can be modeled as follows:

$$\begin{aligned}
 r &= x_j + s_j && \text{for all } j = 1, \dots, k \\
 r &= c + s_{k+1} \\
 z_1 + \dots + z_{k+1} &= 1 \\
 \text{SOS1}(s_j, z_j) &&& \text{for all } j = 1, \dots, k+1 \\
 s_j &\geq 0 && \text{for all } j = 1, \dots, k+1 \\
 z_j &\in \{0, 1\} && \text{for all } j = 1, \dots, k+1
 \end{aligned}$$

The first two constraints state that $r \geq \max\{x_1, \dots, x_k, c\}$, i.e., that the resultant variable r must be at least as large as each of the operand variables x_j and the constant c . This could be modeled using inequalities, but the explicit slack variables play an important role in the constraints that follow.

The next two constraints enforce $r \leq \max\{x_1, \dots, x_k, c\}$, which ensures that r is equal to the MAX expression. Enforcing this side of the equality is actually a lot more complicated. We need to introduce binary auxiliary variables $z_j \in \{0, 1\}$, and SOS1 constraints to required that at most one of the two variables s_j and z_j can be non-zero, which models the implication $z_j = 1 \rightarrow s_j = 0$. Due to the third constraint, one z_j will be equal to 1 and thus at least one s_j will be zero. Hence, $r = x_j$ for at least one j due to the first constraint, or $r = c$ due to the second constraint.

Tolerances play a role in general constraints, although as you might expect, the exact role depends on the constraint type. As a general rule, violations in the resultant will be smaller than the [feasibility tolerance](#), and integrality violations in integer resultants will also satisfy the [integrality tolerance](#).

By most measures, general constraints are just a means of concisely capturing relationships between variables while removing the burden of creating an equivalent MIP formulation. However, general constraints have another potential advantage: Gurobi might be able to simplify the MIP formulation if it can prove during presolve that the simplified version suffices for the correctness of the model. For this reason, Gurobi might be able to produce a smaller or tighter representation of the general constraint than you would get from the most general formulation. For example, it

might be the case that $r \leq \max\{x_1, \dots, x_k, c\}$ is already implied by the other constraints in the model, so that a simple set of inequalities

$$\begin{aligned} r &\geq x_j \quad \text{for all } j = 1, \dots, k \\ r &\geq c \end{aligned}$$

to describe $r \geq \max\{x_1, \dots, x_k, c\}$ suffices to model the relevant part of the MAX constraint.

Norm Constraint

The norm constraint introduces a few complications that are important to be aware of. As mentioned above, this constraint allows you to set one variable equal to the norm of a vector of variables. A few norms are available. The L1 norm is equal to the sum of the absolute values of the operand variables. The L-infinity norm is equal to the maximum absolute value of any operand. The L2 norm is equal to the square root of the sum of the squares of the operands. The L0 norm counts the number of non-zero values among the operands.

Regarding the L2 norm, one obvious complication comes from the fact that enforcing it requires a quadratic constraint. If your model only ever bounds the result from above (e.g., $r = \|x\| \leq 1$), then the resulting constraint will be convex. If your model was otherwise convex, the resulting model will be a (convex) QCP. However, if you try to bound the result from below (e.g., $r = \|x\| \geq y$), adding the L2 norm constraint will lead to a non-convex QCP model, which will typically be significantly harder to solve.

Regarding the L0 norm, note that results obtained with this constraint can be counter-intuitive. This is a consequence of the fact that for nearly any feasible solution with a variable at exactly 0, you can add a small value into that variable while still satisfying all associated constraints to tolerances. The net result is that a lower bound on the L0 norm is often satisfied by “cheating” - by setting enough variables to values that are slightly different from zero. We strongly recommend that you only bound the result from above. That is, you should avoid using the resultant in situations where the model incentivizes a larger value. This would include situations where the objective coefficient is negative, as well as situations where a larger value for the variable could help to satisfy a constraint (e.g., a greater-than constraint where the resultant appears with a positive coefficient).

Function Constraints

Gurobi supports the following function constraints, each with somewhat different syntax and semantics (x and y below are Gurobi decision variables, and other terms are constants provided as input when the constraint is added to the model):

- **Polynomial:** $y = p_0x^n + p_1x^{n-1} + \dots + p_nx + p_{n+1}$
- **Natural exponential:** $y = \exp(x)$ or $y = e^x$
- **Exponential:** $y = a^x$, where $a > 0$ is the base for the exponential function
- **Natural logarithm:** $y = \log_e(x)$ or $y = \ln(x)$
- **Logarithm:** $y = \log_a(x)$, where $a > 0$ is the base for the logarithmic function
- **Logistic:** $y = \frac{1}{1+\exp(-x)}$ or $y = \frac{1}{1+e^{-x}}$
- **Power:** $y = x^a$, where $x \geq 0$ for any a and $x > 0$ for $a < 0$

- **Sine:** $y = \sin(x)$
- **Cosine:** $y = \cos(x)$
- **Tangent:** $y = \tan(x)$

As noted earlier, there is no good way to restate these nonlinear constraints so that a standard MIP solver can handle them efficiently and accurately. The most effective approaches to solving such problems involve replacing the constraints with piecewise-linear approximations. Gurobi provides two different mechanisms for doing so:

- A static piecewise-linear approximation, where the nonlinear function is approximated once, before the MIP solution process begins. That model can then be solved using the standard MIP solver, but the accuracy of the approximation will of course depend heavily on the number of pieces used. For some nonlinear functions, the number of pieces required to achieve the desired accuracy may be lead to excessive solution times.
- A dynamic piecewise-linear approximation, using *outer approximation* and *spatial branch-and-bound*. In this approach, the MIP solver is extended to compute an approximation in the neighborhood of the current relaxation solution at each node of the branch-and-bound search. The solver can compute very accurate solutions using this approach, but having the model change at each node in the branch-and-bound tree greatly complicates the search, which can lead to much less efficient tree exploration.

The most effective approach will depend on your model and your accuracy goals. That's why we provide both options.

Function Constraints with Static Piecewise-Linear Approximation

As noted earlier, the first option for managing function constraints is to perform a static piecewise-linear approximation, yielding a MIP model that can be handed to the standard MIP solver. Gurobi will take this approach for any function where the [FuncNonlinear](#) attribute is set to 0 (or that attribute is set to -1 and the global [FuncNonlinear](#) parameter is set to 0).

With this approach to handling non-linearity, you face a fundamental cost-versus-accuracy tradeoff: adding more pieces produces smaller approximation errors, but also increases the cost of solving the problem. The tradeoff can be complex. Gurobi provides a set of three attributes that help to navigate this tradeoff: [FuncPieces](#), [FuncPieceLength](#), [FuncPieceError](#). They are used as follows:

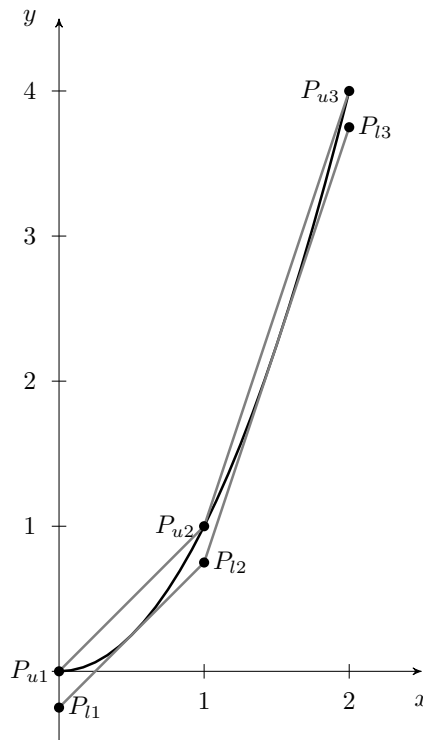
- If you would like to choose the number of pieces to use for the approximation, set the [FuncPieces](#) attribute to the desired value. All pieces will have equal width. This approach allows you to control the size of the approximation.
- If you would like to choose the width of each piece, set the [FuncPieces](#) attribute to a special value of 1 and set the [FuncPieceLength](#) attribute equal to the desired width of each piece. This approach provides some control over both the size and the error of the approximation. While this may appear to be a minor variation of the first option, note that presolve may tighten the domain of x , often substantially, which can make it difficult to predict the relationship between the width of each piece and the number of pieces.

- If you would like to set the maximum error you are willing to tolerate in the approximation, set the `FuncPieces` attribute to a special value of -1 and set the `FuncPieceError` attribute equal to the maximum absolute approximation you are willing to tolerate. Gurobi will choose pieces, typically of different sizes, to achieve that error bound. Note that the number of pieces required may be quite large if you set a tight error tolerance. You can control the maximum relative error rather than the absolute error by setting the `FuncPieces` attribute to -2 instead of -1.

These are attributes on the general constraints, so you can choose different values for each individual constraint.

The other relevant attribute is `FuncPieceRatio`, which controls whether the approximation is an underestimate of the function (0.0), an overestimate (1.0), or somewhere in between (any value strictly between 0.0 and 1.0). You can also choose the special value of -1, which will choose points that are on the original function.

Consider the following simple example:



The goal is to find an approximation of the polynomial $y = x^2$. We've set `FuncPieces` to 1 and `FuncPieceLength` to 1.0, so we're performing an approximation with fixed-width pieces of width 1.0. The domain of x is $[0, 2]$, so the approximation has two pieces. The figure shows 6 points: $P_{u1}(0, 0)$, $P_{u2}(1, 1)$, $P_{u3}(2, 4)$, and $P_{l1}(0, -0.25)$, $P_{l2}(1, 0.75)$, $P_{l3}(2, 3.75)$. If `FuncPieceRatio` is set to 0.0, the approximation would be built from the points below the function (P_{l1} , P_{l2} , and P_{l3}). Similarly, if it is set to 1.0, the approximation would be built from the points above the function (P_{u1} , P_{u2} , and P_{u3}). A value of 0.6 would use weighted combinations of the points: 0.6 times P_{ui}

plus 0.4 times P_{li} . In this case, the line segments would be built from the points $(0, -0.1)$, $(1, 0.9)$, and $(2, 3.9)$. If `FuncPieceRatio` is set to -1 , meaning that the approximation would be built from points that are on the original function, in this case the upper points $(P_{u1}, P_{u2}, \text{ and } P_{u3})$ fit the bill. This will always be the case for a convex function.

Recall that you can set `FuncPieces` to -1 to control the maximum absolute error. In this case, choosing a `FuncPieceError` value of 0.25 would give the piecewise approximation shown in the figure, since the distance between the upper and lower curves is always 0.25. A smaller error value would of course lead to more pieces. We should add that piece widths will typically be non-uniform when limiting the maximum approximation error. The approximation algorithms we use try to limit the number of pieces needed to meet the error targets, which often requires more refinement in some portions of the domain than in others.

Note that the approximations are guaranteed to be under- and over-estimates in all cases except for polynomials of degree greater than 5. Finding the roots of higher-degree polynomials, which would be required to guarantee this property, is quite difficult.

If you wish to experiment with different approaches to approximating a set of functions, it is often convenient to be able to change the approach for all functions at once. We provide a set of parameters with the same names as the attributes to make this easier: `FuncPieces`, `FuncPieceLength`, `FuncPieceError`, and `FuncPieceRatio`. If you set the `FuncPieces` attribute on a function constraint to 0, then the approximation approach for that constraint will be determined by the parameter settings instead.

For some of the supported functions, modest x values can lead to enormous y values (and vice-versa). This can cause numerical issues when solving the resulting piecewise-linear MIP model. To avoid such issues, we limit the range of any x or y that participates in a function constraint to $[-1\text{e}+6, 1\text{e}+6]$. The parameter `FuncMaxVal` allows you to change these limits, but we recommend that you proceed with caution.

We should point out we handle violations and tolerances differently for PWL approximations, which can sometimes lead to unexpected results. For one, the feasibility tolerance for function constraints is specified in `FuncPieceError` rather than through the standard `feasibility tolerance`. We also interpret violations differently. The violation of a function constraint is computed as the Euclidian distance from the solution to the nearest point on the function. Computing this distance exactly can be quite involved for some functions, so we actually compute an over-estimate in some cases. The net result is that the reported violations may be larger than you expect.

Another possible source of unexpected results comes from the fact that solutions that satisfy the original nonlinear function may not satisfy the piecewise-linear approximation of that function. This may lead to sub-optimal solutions or even conclusions of infeasibility. Consider a simple example with two constraints: $y = 2x - 1$ and $y = x^2$. Clearly $(x, y) = (1, 1)$ is a feasible solution, but a piecewise-linear approximation could introduce breakpoints at $x = 0.9$ and $x = 1.1$. The resulting approximation gives a y value of 1.01 at $x = 1$, which is sufficiently far from the actual function value that Gurobi will not consider that a valid solution and declare the model infeasible, since there are no other solutions to the constraints. Reducing the maximum approximation error (by setting `FuncPieces` to -1 and `FuncPieceError` to a much smaller value) would help, but this isn't always the best way to address the problem, since tighter error tolerances can substantially increase the number of pieces in the approximation and thus the cost. We recommend the following approach when you encounter unexpected results. For inequalities, you should ask for an approximation that always overestimates or underestimates the function (depending on the sense of the constraint), to

ensure that your approximation will always satisfy the constraint. The [FuncPieceRatio](#) parameter allows you to do this. For equalities, if you have a sense of where your solution is likely to lie, one option for managing the size of the approximation is to introduce additional variables to capture your function in different ranges, and then perform approximations with different levels of accuracy on these different pieces.

While users could perform piecewise-linear approximations themselves, there are several advantages to asking Gurobi to do it instead. First, Gurobi can often reduce the domains of variables, by using bound strengthening in presolve, or by exploiting repetition in periodic functions like sine or cosine. Smaller domains means fewer pieces to achieve the same accuracy. Gurobi also provides many options to make experimentation easier (for error control, piece length, etc.). These options can be quite difficult to implement and maintain.

Function Constraints With Dynamic Piecewise-Linear Approximation

The alternative for managing function constraints is a more dynamic approach, using *spatial branching* and *outer approximation*. Gurobi will take this approach for any function where the [FuncNonlinear attribute](#) is set to 1 (or that attribute is set to -1 and the global [FuncNonlinear parameter](#) is set to 1).

Solving a model with (nonconvex) nonlinear constraints to global optimality is well known to be a hard task. The idea behind spatial branching is to divide the overall solution space into portions, and to compute valid primal and dual bounds for each portion. Such bounds are obtained by computing a linear approximation to the feasible space for each region (a so-called *outer approximation*) that contains all feasible solutions for the original nonlinear constraints in that region. Bounds for a region can often be tightened by *spatial branching*, where the domain of a variable (integer or continuous) is split, allowing hopefully tighter outer approximations to be computed for the resulting feasible subregions. Bounds for the original model can be obtained by combining the bounds from the leafs of the resulting branch-and-bound tree. This process continues until the desired optimality gap ([MIPGap](#)) is achieved.

Valid primal bounds come from feasible solutions found during the search. They can be found using various heuristics, or they may come from solutions to the current node relaxation that happen to satisfy the nonlinear and integrality constraints (relaxation solutions will always satisfy linear and bound constraints because these are still present in the relaxation). Valid dual bounds are found by solving an LP over a polyhedron that contains all feasible solutions to the original linear and nonlinear constraints in that region (the outer approximation).

Currently, Gurobi only directly supports the univariate nonlinear functions listed in the [Function Constraints](#) section. More complex functions must be disaggregated into a cascade of supported univariate functions. For example, the nonlinear equality constraint

$$x_3 = 100000 \cdot \frac{x_1}{x_1 + 1000 \cdot x_2}$$

can be disaggregated into

$$\begin{aligned} z_1 &= x_1 + 1000 \cdot x_2 \\ z_2 &= z_1^{-1} \\ x_3 &= 100000 \cdot x_1 \cdot z_2. \end{aligned}$$

Such disaggregation is standard practice for most global non-linear solvers (but it is typically performed behind the scenes).

While the disaggregated reformulation is mathematically equivalent to the original model, we should note that the presence of large coefficients in the expression can lead to larger violations of the original constraints than you might expect, due to the accumulation of violations of the disaggregated constraints.

Suppose we are using the default feasibility tolerance of 10^{-6} and Gurobi produces the following solution for the disaggregated model:

$$\begin{aligned}x_1 &= 1 \\x_2 &= 1 \\z_1 &= 1001.000001 \\z_2 &= 0.001000000998003 \\x_3 &= 100.0000998003\end{aligned}$$

As one can see, this solution is indeed feasible within the tolerance for the disaggregated model. However,

$$100000 \cdot \frac{1}{1+1000.1} = 99.9000999001 \neq 100.0000998003 = x_3.$$

The 100000 coefficient magnifies the error in the intermediate result, leading to a violation of almost 0.1 on the original model, which is much larger than the specified feasibility tolerance.

This example illustrates that a solution that is feasible for the disaggregated model may not necessarily be feasible for the aggregated (original) model. Clearly, the above example is an extreme case, but this issue can definitely arise in other situations. This makes scaling and tight coefficient ranges extremely important when formulating and solving nonlinear models.

10.3 Objectives

Every optimization model has an objective function, which is the function on the decision variables that you wish to minimize or maximize. The objective is meant to capture your goals in solving the problem. Given a set of feasible solutions, the objective tells the solver which is preferred.

Most optimization problems have multiple optimal solutions, plus many solutions whose objectives are within a small gap from the optimal value. The solution that is returned by Gurobi depends on the type of problem you are solving. The simple rule is that Gurobi returns a single optimal solution for continuous models (LP, QP, and QCP), and a sequence of improving solutions for discrete models (MIP, MIQP, and MIQCP).

The Gurobi algorithms work on solving a model until they find a solution that is optimal to within the specified tolerances. For the simplex algorithms (including barrier with crossover), the relevant tolerance is the [OptimalityTol](#). For the barrier algorithm (without crossover), the relevant tolerances are the [BarConvTol](#) or [BarQCPCConvTol](#) (depending on the problem type). You can relax these tolerances, but note that it is rare for this to significantly improve solution times. The simplex and barrier algorithms both return a single optimal solution.

For discrete models, while you can ask the MIP solver to find a solution with the best possible objective value, it is more common to stop when the solution objective is within a specified gap from the optimal value. This optimality gap is controlled by the [MIPGap](#) parameter, and the default value is 0.01%.

The MIP solver typically finds multiple sub-optimal solutions on the way to eventually finding an optimal solution. These intermediate solutions can be queried once the optimization is complete

(using the [Xn](#) attribute). You can use the [Solution Pool](#) feature to take a more systematic approach to finding multiple solutions. This feature allows you to indicate how many solutions you would like, to specify the largest gap to the optimal value you are willing to accept, etc.

We should add that it is possible to specify a pure feasibility problem, where the sole goal is to find a solution that satisfies the constraints. You can think of a feasibility problem as an optimization problem with a constant objective function.

The available objective types are [linear](#), [piecewise-linear](#), [quadratic](#) (both convex and non-convex), and [multi-objective](#). While the property of having multiple objectives may appear to be orthogonal to the types of the objectives, Gurobi only supports multi-objective models where all objectives are linear.

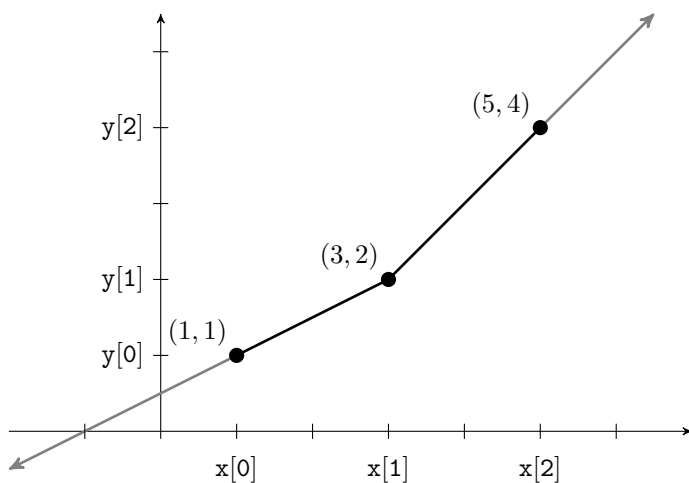
Linear Objectives

The simplest and most common objective function is linear - minimizing or maximizing a linear function on the decision variables (e.g., $3x + 4y + 2$). Linear objectives can be specified in a few ways. The first is by providing an objective term when the decision variable is added to the model (typically through the `addVar` method). The second is by setting the [Obj](#) attribute on the variable. The third and most convenient approach, available in the object-oriented interfaces, is through the `setObjective` method (in [C++](#), [Java](#), [.NET](#), or [Python](#)). This method accepts a linear expression object as its argument.

A model with a linear objective, only linear constraints, and only continuous variables is a Linear Program (LP). It can be solved using the simplex or barrier algorithms.

Piecewise-Linear Objectives

A useful variant of a linear objective is a *piecewise-linear* objective, where the objective for a single variable is captured in a set of linear pieces. For example, suppose we want to define the objective value $f(x)$ for variable x as follows:



The vertices of the function occur at the points (1, 1), (3, 2) and (5, 4), so we define $f(1) = 1$, $f(3) = 2$ and $f(5) = 4$. Other objective values are linearly interpolated between neighboring points. The first pair and last pair of points each define a ray, so values outside the specified x values are extrapolated from these points. Thus, in our example, $f(-1) = 0$ and $f(6) = 5$.

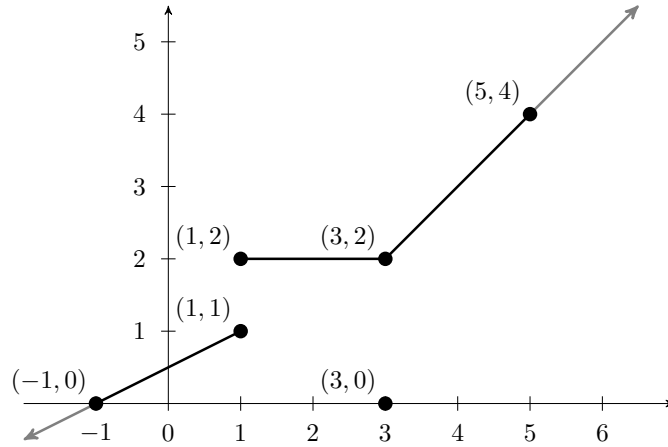
More formally, a piecewise-linear function is defined by a set of n points:

$$\mathbf{x} = [x_1, \dots, x_n], \quad \mathbf{y} = [y_1, \dots, y_n]$$

These define the following piecewise-linear function:

$$f(v) = \begin{cases} y_1 + \frac{y_2 - y_1}{x_2 - x_1}(v - x_1), & \text{if } v \leq x_1, \\ y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(v - x_i), & \text{if } v \geq x_i \text{ and } v \leq x_{i+1}, \\ y_n + \frac{y_n - y_{n-1}}{x_n - x_{n-1}}(v - x_n), & \text{if } v \geq x_n. \end{cases}$$

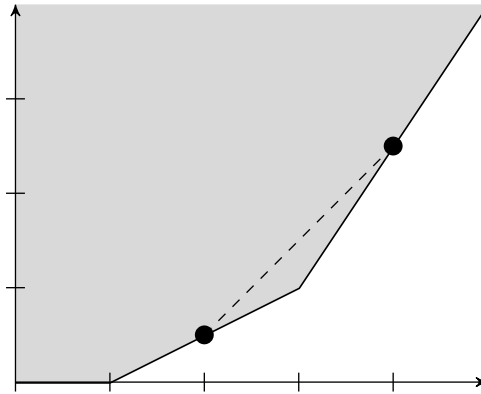
We also allow special cases, such as jumps and single points, which are quite useful to define the fixed charges or the penalties. A jump at $x = x_i$ means that the left piece and the right piece don't intersect at $x = x_i$, i.e. we have $(x_{i-1}, y_{i-1}), (x_i, y_i), (x_{i+1}, y_{i+1}), (x_{i+2}, y_{i+2})$ with $x_i = x_{i+1}$ and $y_i \neq y_{i+1}$. So for the left piece, i.e. $x_{i-1} \leq x < x_i$, the line segment between points (x_{i-1}, y_{i-1}) and (x_i, y_i) defines y , for the right piece, i.e. $x_i \leq x < x_{i+2}$, the line segment between points (x_{i+1}, y_{i+1}) and (x_{i+2}, y_{i+2}) defines y . Since we must allow some tolerance for numeric computation, it means that at $x = x_i$, y can take the value of either y_i or y_{i+1} . A single point at $x = x_i$ means that both left and right pieces extend to $x = x_i$, but both have different y values than y_i . It can be described by the five points $(x_{i-2}, y_{i-2}), (x_{i-1}, y_{i-1}), (x_i, y_i), (x_{i+1}, y_{i+1}), (x_{i+2}, y_{i+2})$ with $x_{i-1} = x_i = x_{i+1}$ and $y_i \neq y_{i-1}$ and $y_i \neq y_{i+1}$. Note that y_{i-1} and y_{i+1} can be equal or different. Because of the tolerance, it means that at $x = x_i$, y can take the value of y_{i-1} , y_i or y_{i+1} . Here below is an example with a jump and a single point.



The above piecewise function for variable x are defined by 7 points $(-1, 0), (1, 1), (1, 2), (3, 2), (3, 0), (3, 2)$ and $(5, 4)$. It has a jump at $x = 1$ from $(1, 1)$ to $(1, 2)$ and a single point $(3, 0)$. Note that both left and right points have the same x coordinate and for this example the two points are the same.

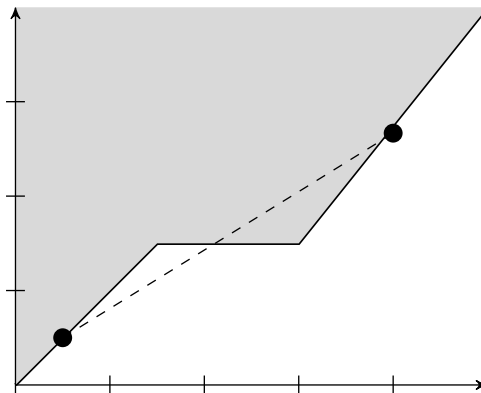
Note that a piecewise-linear objective can change the type of a model. Specifically, including a non-convex piecewise-linear objective function in a continuous model will transform that model into a MIP. This can significantly increase the cost of solving the model.

How do you determine whether your piecewise-linear objective is convex? A convex function has the property that you can't obtain a better objective value by interpolating between two points on the function. In the figure below, you will note that all convex combinations of points on the piecewise-linear function are in the shaded (feasible) region.



Stated another way, successive pieces will have non-decreasing slopes in a convex piecewise-linear objective (assuming you are minimizing).

In contrast, in a non-convex piecewise-linear function you can get a better value by interpolating between points. In the figure below, the value of $f(1)$ for the piecewise-linear function is worse (larger) than the value obtained by interpolation.



Piecewise-linear objectives are defined on variables using a special method (in [C](#), [C++](#), [Java](#), [.NET](#), or [Python](#)). Each variable can have its own piecewise-linear objective function, and each function requires a separate call to this method.

A variable can't have both a linear and a piecewise-linear objective term. Setting a piecewise-linear objective for a variable will set the [Obj attribute](#) on that variable to 0. Similarly, setting the [Obj attribute](#) will delete the piecewise-linear objective on that variable.

We should point out that it is fairly easy to specify a piecewise-linear function on a variable using a few extra variables and simple linear objective terms. The advantages of using the piecewise-linear

API methods are twofold. The first is convenience - specifying the function directly leads to simpler and more readable code. The second is that Gurobi includes a piecewise-linear simplex algorithm. If you provide a model that contains only linear constraints, only continuous variables, and only linear or convex piecewise-linear objective terms, then this specialized simplex algorithm will be used to solve the model. If your piecewise-linear function contains a large number of segments, the specialized algorithm will be much faster than the standard simplex solver.

Quadratic Objectives

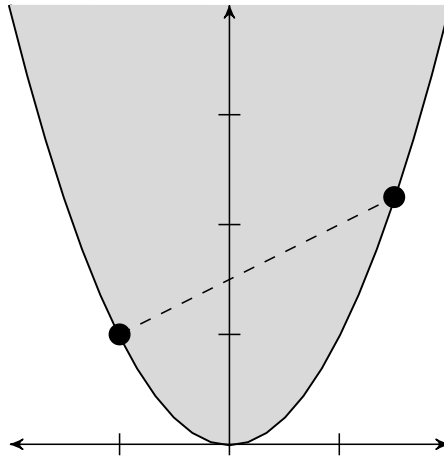
Your optimization objective can also contain quadratic terms (e.g., $3x^2 + 4y^2 + 2xy + 2x + 3$). You specify quadratic objectives in the object-oriented interfaces by building quadratic expressions and then calling *setObjective* ([C++](#), [Java](#), [.NET](#), or [Python](#)). In C, you input your quadratic terms using [GRBaddqterms](#).

There are four distinct algorithms that could be used to solve a model with a quadratic objective. The appropriate one depends on a few specific properties of the objective and the rest of the model.

- **Continuous Convex QP** If your quadratic objective is convex and your model only contains linear constraints and continuous variables, then your model is a quadratic program (QP) and can be solved using either the simplex or barrier algorithms. QP simplex will return an optimal basic solution. Gurobi does not have a QP crossover, so QP barrier will return an interior solution.
- **Discrete QP with Convex Relaxation** If your quadratic objective is convex but the model contains discrete elements (integer variables, SOS constraints, general constraints, etc.), then your model is a mixed integer quadratic program (MIQP) and is solved using the MIP solver. Since MIP relies heavily on simplex bases, the root relaxation must be solved using the primal or dual simplex algorithm.
- **Non-convex QP and Discrete QP with Non-Convex Relaxation** If your quadratic objective is not convex, then the model will be solved using the MIP solver, even if your model has no explicit discrete elements. Non-convex problems often lead to much larger solve times, and it might be that the non-convexity in your model is unexpected, for example due to an error in the model data or in the model formulation. If you think that your model should be convex (or, in the discrete case, have a convex relaxation), you can set the [NonConvex](#) parameter to 0 or 1. With this setting, a non-convex quadratic objective leads to a `GRB_ERROR_Q_NOT_PSD` error.

These properties are checked on the presolved model. As is always the case, presolve will try to simplify the model. In this context, it will try to transform a non-convex MIQP into an equivalent convex MIQP. This simplification will always succeed if each quadratic term contains at least one binary variable.

How can you determine whether your quadratic objective is convex? As was noted earlier, the crucial property for convexity is that interpolation between any two points on the function never puts you below the function (assuming minimization). In this figure, all points on a line segment between any two points on the parabola are always in the shaded region.



How does this translate to multiple variables? For a quadratic function to be convex, the associated Q matrix must be Positive Semi-Definite (PSD).

Multiple Objectives

You can also specify multiple (linear) objectives for your model, and Gurobi provides tools that allow you explore the tradeoffs between them. Refer to the [Multiple Objectives](#) section for additional details.

10.4 Tolerances and Ill Conditioning -- A Caveat

As noted at several places in this section, finite-precision arithmetic limits the precision of the solutions Gurobi computes. This limitation is managed through numerical tolerances in most cases; we treat a solution as satisfying a constraint if the violation is smaller than the corresponding tolerance. The default tolerances are chosen to be sufficiently large so that numerical errors aren't an issue for most models, yet small enough that the results of the computations are meaningful.

Unfortunately, some models suffer from severe *ill conditioning*, which can greatly complicate the search for a solution. This can show itself in a few ways. Ill conditioning can severely hurt performance, and it can lead to solutions whose constraint violations are larger than the tolerances.

Ill conditioning is a measure of the amount of error that can result when solving linear systems of equations. As noted earlier, linear and mixed-integer programming are built on top of linear solves, so errors in solving linear systems directly lead to errors in LP and MIP solutions. Serious problems arise when the error in solving a linear system is comparable to the desired tolerance. If you want to solve a linear programming problem to the default feasibility tolerance of $1e-6$, for example, and if your linear system solves produce errors that are also roughly $1e-6$, then you have no way of knowing whether your current solution is truly feasible. This can lead to oscillations, as your solution bounces between feasible and infeasible due to nothing more than numerical error, which can make it extremely difficult to achieve forward progress towards an optimal solution.

When solving linear and quadratic programming problems, we recommend that you check final primal and dual constraint violations. Duality theory states that, if your solution is primal feasible, dual feasible, and complementary, then you have an optimal solution. Complementarity is automatically enforced by the simplex method, so achieving primal and dual feasibility (to tolerances) assures that the solution is optimal (to tolerances).

When solving a MIP model (which includes any model that contains discrete or non-convex features, such as non-convex objectives, general constraints, semi-continuous variables, etc.), there is unfortunately no simple method available to check the optimality of the result. While we work hard to identify and manage the negative effects of ill conditioning, we are unable to provide a mathematical proof that the solution returned is truly optimal.

For additional information on numerical issues, please refer to the [Gurobi Guidelines for Numerical Issues](#) section of this manual.